UNIVERSIDAD NACIONAL JOSÉ MARÍA ARGUEDAS

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

MANUAL TÉCNICO

Proyecto:

“CHANKA STUDIO: EDITOR DE DISEÑO TEXTIL 2D”

Presentado por:

Aguila Silvera Juan David

Quispe Andrada Betsaly Mayumi

Sotomayor Medina Shani Nahir

Docente:

Rojas Silva Jhamer Anderson

ANDAHUAYLAS – APURÍMAC

PERÚ – 2026

1. PROPÓSITO Y ALCANCE

Este manual documenta la organización técnica de Chanka Studio, una aplicación web para diseño textil 2D. Está dirigido a desarrolladores que necesiten ejecutar, revisar, mantener o ampliar el código. Se describen las tecnologías identificadas en el proyecto, la arquitectura modular, el pipeline gráfico, los algoritmos principales, las instrucciones de ejecución y las pruebas disponibles.

La aplicación utiliza scripts JavaScript clásicos que exponen componentes mediante el objeto global Chanka. La página de entrada es index.html. La versión exacta de Node.js y del navegador debe registrarse en el equipo de entrega, porque no queda fijada en la documentación disponible.

2. TECNOLOGÍAS Y HERRAMIENTAS

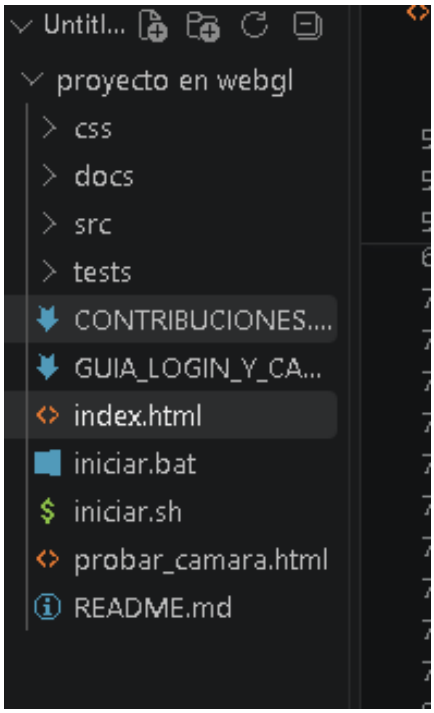
TECNOLOGÍA	USO	VERSIÓN / OBSERVACIÓN
HTML5	Estructura de la página y controles de interfaz.	Estándar web; registrar versión del navegador.
CSS	Estilo y distribución visual.	Archivo css/estilos.css.
JavaScript	Algoritmos de curvas, matrices, estado del editor, eventos y archivos.	Scripts clásicos; registrar navegador y Node.js.
SVG	Representación vectorial de prendas, motivos y segmentos.	Lienzo con viewBox de 800 × 800.
Node.js	Ejecución de pruebas automatizadas.	Registrar la versión instalada.
Visual Studio Code (opcional)	Edición del código fuente.	Registrar versión si se utilizó.

No se debe indicar una versión concreta si no se ha comprobado en el entorno real. Para registrar las versiones, ejecutar node --version en la terminal y consultar la sección Acerca de del navegador utilizado.

3. ARQUITECTURA MODULAR

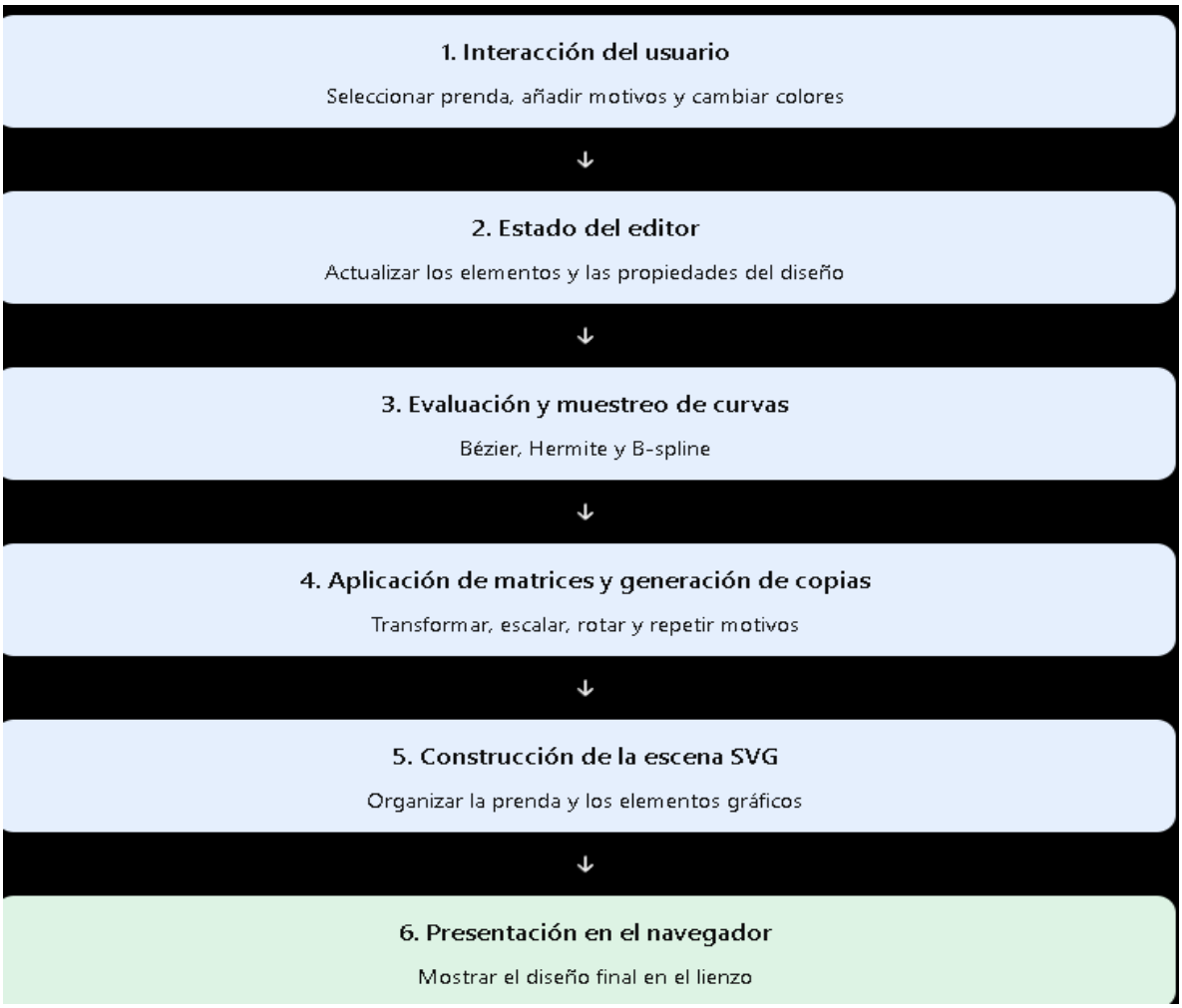
La arquitectura separa la entrada de la aplicación, el cálculo matemático, la geometría, el renderizado, la lógica del editor y los controles de interfaz. Las dependencias principales siguen el flujo curvas → geometría → renderizado/editor → interfaz. La entrada index.html carga los recursos y src/main.js inicia Chanka.Panel.iniciar() cuando el DOM está listo.

MÓDULO	ARCHIVOS PRINCIPALES	RESPONSABILIDAD
Entrada	index.html; src/main.js	Carga la página e inicia el panel cuando el DOM está listo.
Curvas	src/curvas/bezier.js; hermite.js; bspline.js	Evalúa puntos de curvas paramétricas.
Geometría	src/geometria/matrices.js; trazo.js	Transformaciones, muestreo, suavizado, repeticiones y pruebas geométricas.
Renderizado	src/render/prendas.js; escena.js	Define el catálogo de prendas y dibuja la escena en SVG.
Editor y archivos	src/editor/motivos.js; plantillas.js; editor.js; archivo.js	Define motivos y plantillas; gestiona estado, historial, validación, guardado, apertura y exportación.
Interfaz	src/ui/panel.js	Conecta controles y acciones del usuario con el editor y la escena.



4. PIPELINE GRÁFICO

El pipeline describe cómo una acción del usuario termina representada visualmente. El flujo documentado es: evento de interfaz → actualización del estado del editor → muestreo de la curva (100 pasos y 101 puntos por curva) → transformación de puntos mediante matrices → generación de repeticiones y espejo → composición de escena → dibujo de segmentos SVG → actualización visual.



5. ALGORITMOS Y FÓRMULAS

5.1 Curvas Bézier: algoritmo de De Casteljau

Archivo: src/curvas/bezier.js. La función punto(p, t) copia los puntos de control y realiza interpolaciones lineales sucesivas entre puntos vecinos. Para cada nivel r se calcula:

$$P_i^{(r)}(t) = (1-t)P_i^{(r-1)}(t) + tP_{i+1}^{(r-1)}(t)$$

El proceso continúa hasta obtener un único punto perteneciente a la curva. Se utiliza por su formulación directa y porque permite calcular la curva sin delegar todo el trabajo a un motor gráfico externo.

src > curvas > JS bezier.js > ...

```
// CURVA DE BÉZIER — algoritmo de De Casteljau
// p = lista de puntos de control [{x,y}, ...]   t = número entre 0 y 1
// Idea: en cada nivel se mezclan los puntos vecinos hasta que queda uno solo.
(function (G) {
  const Chanka = (G.Chanka = G.Chanka || {});

  function punto(p, t) {
    let q = p.map(v => ({ x: v.x, y: v.y }));
    for (let nivel = q.length - 1; nivel > 0; nivel--) {
      for (let i = 0; i < nivel; i++) {
        q[i] = {
          x: (1 - t) * q[i].x + t * q[i + 1].x,
          y: (1 - t) * q[i].y + t * q[i + 1].y,
        };
      }
    }
    return q[0];
  }

  Chanka.Bezier = { punto };
})(typeof window !== 'undefined' ? window : globalThis);

```

5.2 Curvas Hermite

Archivo: src/curvas/hermite.js. El algoritmo recibe los extremos y dos manejadores. Las tangentes se calculan como diferencias entre los manejadores y los extremos, y se combinan con las funciones base de Hermite:

$H(t) = h_{00}P_0 + h_{01}P_1 + h_{10}T_0 + h_{11}T_1$

$h_{00} = 2t^3 - 3t^2 + 1$; $h_{01} = -2t^3 + 3t^2$; $h_{10} = t^3 - 2t^2 + t$; $h_{11} = t^3 - t^2$.

src > curvas > JS hermite.js > ...

```
// CURVA DE HERMITE
// p = [inicio, fin, manejadorInicio, manejadorFin]
// Los manejadores marcan hacia dónde sale y llega la curva (las tangen
(function (G) {
  const Chanka = (G.Chanka = G.Chanka || {});

  function punto(p, t) {
    const t2 = t * t, t3 = t2 * t;
    // Las 4 funciones base de Hermite
    const h00 = 2 * t3 - 3 * t2 + 1;
    const h01 = -2 * t3 + 3 * t2;
    const h10 = t3 - 2 * t2 + t;
    const h11 = t3 - t2;

    const inicio = p[0], fin = p[1];
    const tan0 = { x: p[2].x - inicio.x, y: p[2].y - inicio.y };
    const tan1 = { x: p[3].x - fin.x, y: p[3].y - fin.y };

    return {
      x: h00 * inicio.x + h01 * fin.x + h10 * tan0.x + h11 * tan1.x,
      y: h00 * inicio.y + h01 * fin.y + h10 * tan0.y + h11 * tan1.y,
    };
  }

  Chanka.Hermite = { punto };
})(typeof window !== 'undefined' ? window : globalThis);

```

5.3 Curvas B-spline: algoritmo de De Boor

Archivo: src/curvas/bspline.js. El módulo determina el grado, genera el vector de nudos, localiza el intervalo del parámetro t y combina los puntos de control por niveles. Para seis puntos de control y grado tres, la guía técnica registra nudos abiertos [0, 0, 0, 0, 1/3, 2/3, 1, 1, 1, 1]. Se utiliza porque permite controlar la forma mediante puntos de control y producir curvas suaves.

src > curvas > JS bspline.js > ...

```
// B-SPLINE CÚBICA — algoritmo de De Boor
// Con 6 puntos de control y nudos "abiertos" la curva empieza en el pr
// punto y termina en el último; los del medio solo la "jalan".
(function (G) {
  const Chanka = (G.Chanka = G.Chanka || {});

  // Vector de nudos abierto uniforme. Ejemplo con 6 puntos: [0,0,0,0, 1/3, 2/3,
  function nudos(n, grado) {
    const u = [];
    for (let i = 0; i < n + grado + 1; i++) {
      if (i <= grado) u.push(0);
      else if (i >= n) u.push(1);
      else u.push((i - grado) / (n - grado));
    }
    return u;
  }

  function punto(p, t) {
    const n = p.length - 1;
    const grado = Math.min(3, n);
    const u = nudos(p.length, grado);

    // 1) buscar en qué tramo de nudos está t
    let k = grado;
    if (t >= 1) k = n;
    else while (k < n && t >= u[k + 1]) k++;

    // 2) copiar los puntos que influyen y mezclarlos nivel por nivel
    const d = [];
    for (let j = 0; j <= grado; j++) d.push({ ...p[k - grado + j] });
    for (let r = 1; r <= grado; r++) {
      for (let j = grado; j >= r; j--) {
        const i = k - grado + j;
        const den = u[i + grado - r + 1] - u[i];
        const a = den ? (t - u[i]) / den : 0;
        d[j] = {
          x: (1 - a) * d[j - 1].x + a * d[j].x,
          y: (1 - a) * d[j - 1].y + a * d[j].y,
        };
      }
    }
    return d[grado];
  }

  Chanka.Bspline = { punto, nudos };
})(typeof window !== 'undefined' ? window : globalThis);

```

5.4 Transformaciones geométricas con matrices

Archivo: src/geometria/matrices.js. La matriz compacta [a, b, c, d, e, f] representa la transformación homogénea:

$$[a \quad c \quad e]$$

$$[b \quad d \quad f]$$

$$[0 \quad 0 \quad 1]$$

La transformación de un punto es x′ = ax + cy + e, y′ = bx + dy + f. El módulo implementa identidad, traslación, escalado, rotación, multiplicación, aplicación e inversa de matrices. La composición está definida de modo que, al calcular multiplicar(A, B), B se aplica primero y A después. La inversa permite transformar coordenadas y facilitar la edición en el sistema local de un motivo.

src > geometria > JS matrices.js > ...

```
// TRANSFORMACIONES 2D CON MATRICES
// Una matriz es una lista [a, b, c, d, e, f] que significa:
//     x' = a·x + c·y + e
//     y' = b·x + d·y + f
// (equivale a la matriz 3x3 [[a c e],[b d f],[0 0 1]])
(function (G) {
  const Chanka = (G.Chanka = G.Chanka || {});

  const identidad = () => [1, 0, 0, 1, 0, 0];
  const trasladar = (x, y) => [1, 0, 0, 1, x, y];
  const escalar = (sx, sy = sx) => [sx, 0, 0, sy, 0, 0];

  function rotar(grados) {
    const r = (grados * Math.PI) / 180, c = Math.cos(r), s = Math.sin(r);
    return [c, s, -s, c, 0, 0];
  }

  // A·B : primero se aplica B y después A
  function multiplicar(A, B) {
    return [
      A[0] * B[0] + A[2] * B[1],
      A[1] * B[0] + A[3] * B[1],
      A[0] * B[2] + A[2] * B[3],
      A[1] * B[2] + A[3] * B[3],
      A[0] * B[4] + A[2] * B[5] + A[4],
      A[1] * B[4] + A[3] * B[5] + A[5],
    ];
  }

  function aplicar(M, p) {
    return { x: M[0] * p.x + M[2] * p.y + M[4], y: M[1] * p.x + M[3] *
  }

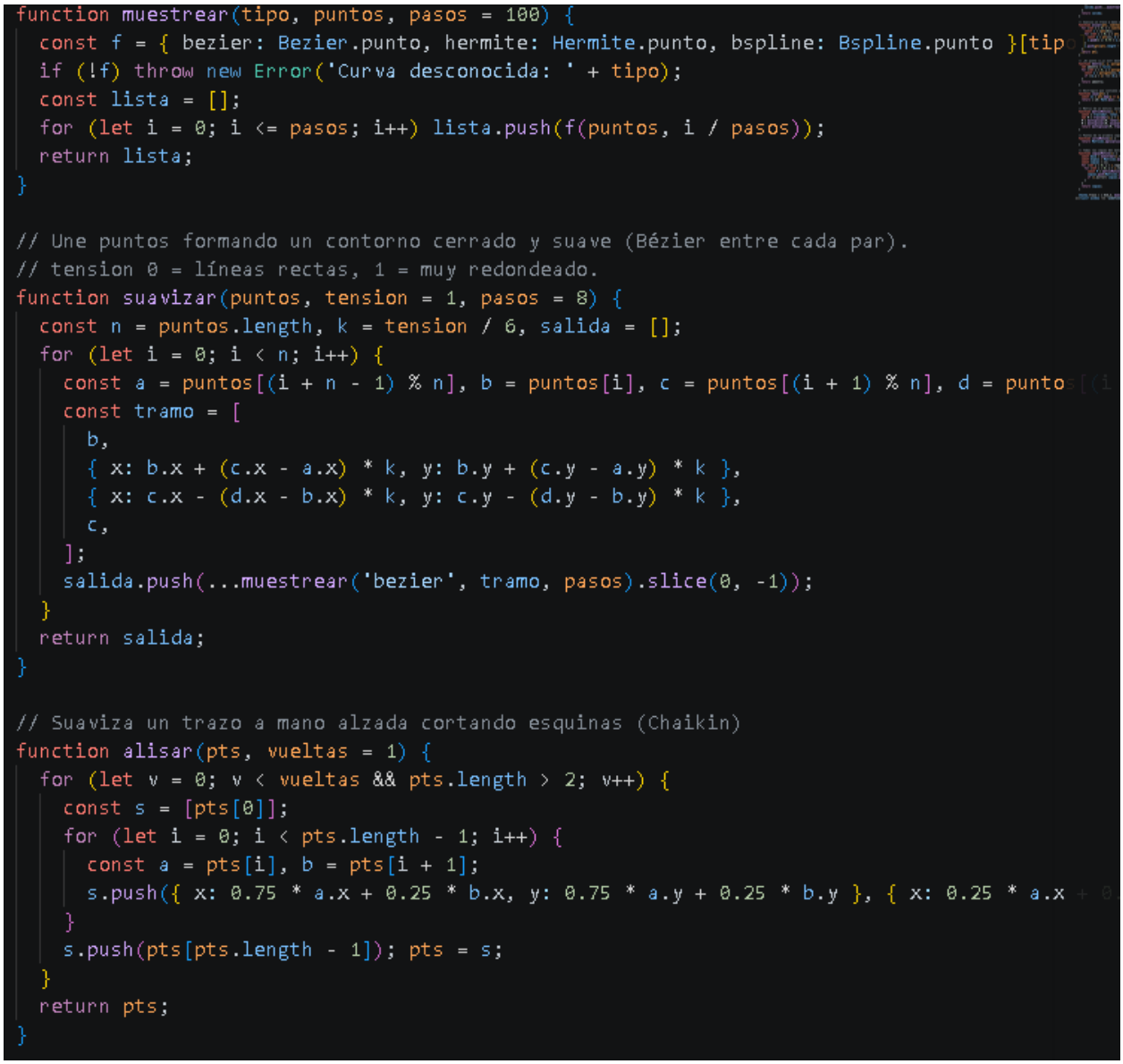
  const aplicarLista = (M, lista) => lista.map(p => aplicar(M, p));

  // Matriz inversa: sirve para pasar de la pantalla al sistema del mot
  function inversa(M) {
    const det = M[0] * M[3] - M[1] * M[2];
    const a = M[3] / det, b = -M[1] / det, c = -M[2] / det, d = M[0] /
    return [a, b, c, d, -(a * M[4] + c * M[5]), -(b * M[4] + d * M[5])];
  }

  Chanka.Matrices = { identidad, trasladar, escalar, rotar, multiplicar,
})(typeof window !== 'undefined' ? window : globalThis);
```

5.5 Muestreo, suavizado, interior de polígonos y copias

Archivo: src/geometria/trazo.js. La función muestrear() evalúa la curva en 101 parámetros para 100 pasos. suavizar() forma contornos mediante segmentos Bézier; alisar() utiliza una variante del método de Chaikin para suavizar trazos. dentro() emplea el método de cruce de rayos para determinar si un punto está dentro de un polígono. matrizMotivo() compone traslación, rotación, escala y traslado del pivote. copiasMotivo() genera repeticiones y puede aplicar espejo respecto al eje vertical central x = 400.



```
function muestrear(tipo, puntos, pasos = 100) {
  const f = { bezier: Bezier.punto, hermite: Hermite.punto, bspline: Bspline.punto }[tip
  if (!f) throw new Error('Curva desconocida: ' + tipo);
  const lista = [];
  for (let i = 0; i <= pasos; i++) lista.push(f(puntos, i / pasos));
  return lista;
}

// Une puntos formando un contorno cerrado y suave (Bézier entre cada par).
// tension 0 = líneas rectas, 1 = muy redondeado.
function suavizar(puntos, tension = 1, pasos = 8) {
  const n = puntos.length, k = tension / 6, salida = [];
  for (let i = 0; i < n; i++) {
    const a = puntos[(i + n - 1) % n], b = puntos[i], c = puntos[(i + 1) % n], d = punto
    const tramo = [
      b,
      { x: b.x + (c.x - a.x) * k, y: b.y + (c.y - a.y) * k },
      { x: c.x - (d.x - b.x) * k, y: c.y - (d.y - b.y) * k },
      c,
    ];
    salida.push(...muestrear('bezier', tramo, pasos).slice(0, -1));
  }
  return salida;
}

// Suaviza un trazo a mano alzada cortando esquinas (Chaikin)
function alisar(pts, vueltas = 1) {
  for (let v = 0; v < vueltas && pts.length > 2; v++) {
    const s = [pts[0]];
    for (let i = 0; i < pts.length - 1; i++) {
      const a = pts[i], b = pts[i + 1];
      s.push({ x: 0.75 * a.x + 0.25 * b.x, y: 0.75 * a.y + 0.25 * b.y }, { x: 0.25 * a.x +
    }
    s.push(pts[pts.length - 1]); pts = s;
  }
  return pts;
}
```

```
// ¿El punto (x,y) está dentro del polígono? (cruce de rayos)
function dentro(x, y, poligono) {
  let adentro = false;
  for (let i = 0, j = poligono.length - 1; i < poligono.length; j = i++) {
    const a = poligono[i], b = poligono[j];
    if ((a.y > y) !== (b.y > y) && x < ((b.x - a.x) * (y - a.y)) / (b.y - a.y) + a.x) ad
  }
  return adentro;
}

// Rectángulo que contiene a todos los puntos
function caja(pts) {
  const xs = pts.map(p => p.x), ys = pts.map(p => p.y);
  return { x0: Math.min(...xs), y0: Math.min(...ys), x1: Math.max(...xs), y1: Math.max(...ys) };
}

// Matriz de un motivo: mover al centro, girar y escalar alrededor de su pivote (cx, cy)
function matrizMotivo(c, i = 0, j = 0) {
  const { trasladar, rotar, escalar, multiplicar } = Matrices;
  let M = trasladar(c.x + i * c.gapX, c.y + j * c.gapY);
  M = multiplicar(M, rotar(c.rot || 0));
  M = multiplicar(M, escalar(c.scale));
  return multiplicar(M, trasladar(-c.cx, -c.cy));
}

// Puntos de la primera copia del motivo, ya en coordenadas del lienzo
function curvaMotivo(c) {
  return Matrices.aplicarLista(matrizMotivo(c, 0, 0), muestrear(c.type, c.points));
}

// Todas las copias del motivo (repeticiones + espejo)
function copiasMotivo(c) {
  const base = muestrear(c.type, c.points);
  const espejo = Matrices.multiplicar(Matrices.trasladar(2 * EJE_X, 0), Matrices.escalar(-1, 1));
  const copias = [];
  for (let j = 0; j < c.repeatY; j++) {
    for (let i = 0; i < c.repeatX; i++) {
      const M = matrizMotivo(c, i, j);
      copias.push(Matrices.aplicarLista(M, base));
      if (c.mirror) copias.push(Matrices.aplicarLista(Matrices.multiplicar(espejo, M), base));
    }
  }
  return copias;
}
```

6. RENDERIZADO, DATOS Y ESTADO

6.1 Prendas y escena

src/render/prendas.js almacena los puntos y detalles de las prendas dentro del lienzo 800 × 800. src/render/escena.js representa la prenda, los motivos y las líneas calculadas mediante elementos SVG. El cálculo de las curvas ocurre en los módulos propios; el renderizador dibuja los segmentos ya muestreados.

6.2 Modelo de datos y persistencia

El diseño se serializa como JSON e incluye campos como nombre, identificador, prenda, color, contorno, fotografía de referencia si existe, modo de vista, motivos y trazos de pintura. Archivo.validar() comprueba estructura, tipos, rangos y cantidades antes de abrir. Archivo.migrar() contempla la conversión de diseños de la versión 1. La interfaz también puede mantener diseños en el almacenamiento local del navegador.

6.3 Plantillas, pintura e historial

src/editor/plantillas.js crea motivos desde configuraciones predefinidas. El editor conserva un historial para deshacer y rehacer. Los trazos de pintura almacenan identificador, color, grosor, opacidad y puntos; el borrador elimina el trazo seleccionado. Los trazos se dibujan recortados a la prenda y debajo de los motivos.

src > editor > JS plantillas.js > <function> > lista > crear

```
// PLANTILLAS — diseños de partida: un clic arma una composición lista para editar
// Cada plantilla es solo datos + una función que devuelve motivos (usa los de motivos.js).
(function (G) {
  const Chanka = (G.Chanka = G.Chanka || {});
  const { Motivos } = Chanka;
  const indice = nombre => Motivos.PRESETS.findIndex(p => p.nombre === nombre);

  // Fila de n copias de un motivo, centrada en el lienzo (x = 400)
  function fila(nombre, y, n, sep, op = {}) {
    const c = Motivos.desdePreset(indice(nombre), 400 - ((n - 1) * sep) / 2, y, { color: op.color });
    c.repeatX = n; c.gapX = sep; c.scale = op.escala || 1;
    return c;
  }
  function simetrico(nombre, x, y, color) {
    const c = Motivos.desdePreset(indice(nombre), x, y, { color });
    c.mirror = true;
    return c;
  }

  const lista = [
    { id: 'blanco', nombre: 'En blanco', crear: () => [] },
    { id: 'franjas', nombre: 'Franjas andinas', base: '#e6d7b9', crear: () => [
      fila('Franja', 250, 4, 150, { color: '#8d3439', escala: 1.1 }),
      fila('Picos', 420, 5, 120, { color: '#3c5663' }),
      fila('Franja', 590, 4, 150, { color: '#8d3439', escala: 1.1 })] },
    { id: 'borde', nombre: 'Borde de picos', base: '#373733', crear: () => [
      fila('Onda', 560, 5, 140, { color: '#bb9148' }),
      fila('Picos', 640, 6, 115, { color: '#f3ce7e', escala: 0.9 })] },
    { id: 'gotas', nombre: 'Campo de gotas', base: '#6c7458', crear: () => {
      const g = fila('Gota', 300, 4, 130, { color: '#e6d7b9' });
      g.repeatY = 3; g.gapY = 130;
      return [g];
    } },
    { id: 'simetrico', nombre: 'Centro simétrico', base: '#a35f46', crear: () => [
      simetrico('Media luna', 300, 400, '#f3ce7e'), simetrico('Gancho', 250, 260, '#e6d7b9')] },
    { id: 'olas', nombre: 'Olas verticales', base: '#3c5663', crear: () => [
      fila('Serpentina', 300, 3, 200, { color: '#e6d7b9' }),
      fila('Serpentina', 480, 3, 200, { color: '#f3ce7e' })] },
  ];

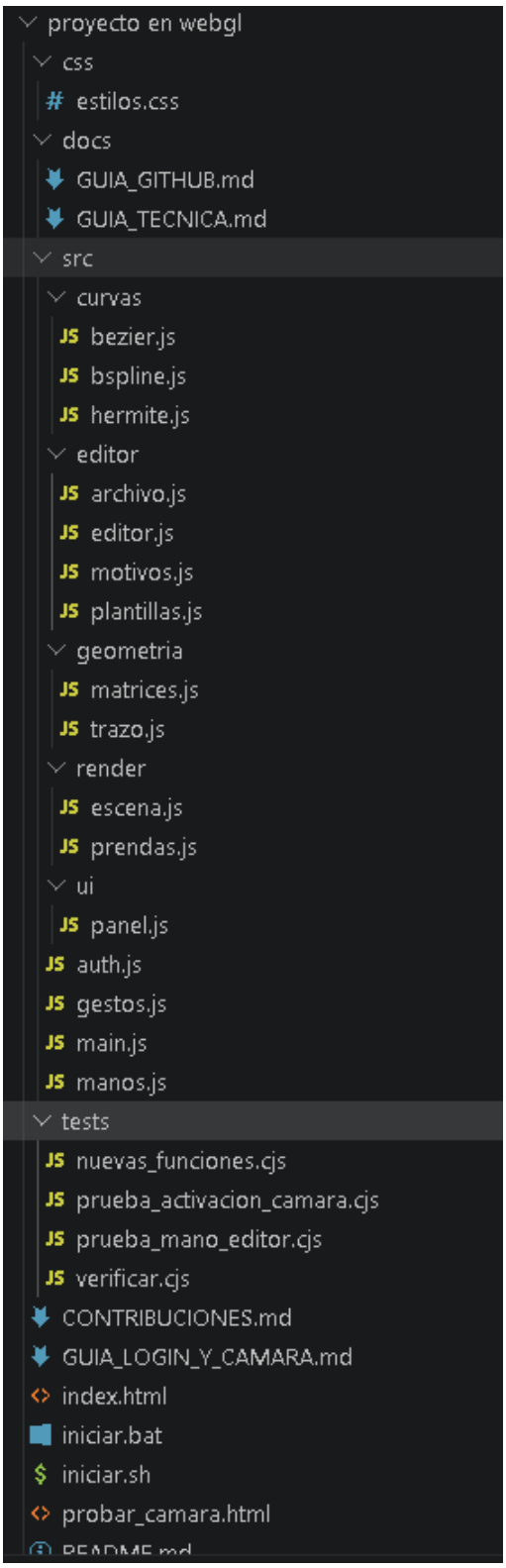
  Chanka.Plantillas = { lista };
})(typeof window !== 'undefined' ? window : globalThis);

```

7. ESTRUCTURA DEL REPOSITORIO

La estructura descrita en el paquete del proyecto es la siguiente:

- index.html
- css/estilos.css
- src/main.js
- src/curvas/bezier.js
- src/curvas/hermite.js
- src/curvas/bspline.js
- src/geometria/matrices.js
- src/geometria/trazo.js
- src/render/prendas.js
- src/render/escena.js
- src/editor/motivos.js
- src/editor/plantillas.js
- src/editor/editor.js
- src/editor/archivo.js
- src/ui/panel.js
- tests/verificar.cjs
- docs/GUIA_TECNICA.md
- docs/GUIA_GITHUB.md
- README.md
- CONTRIBUCIONES.md



8. EJECUCIÓN DESDE EL CÓDIGO FUENTE

8.1 Ejecutar la aplicación

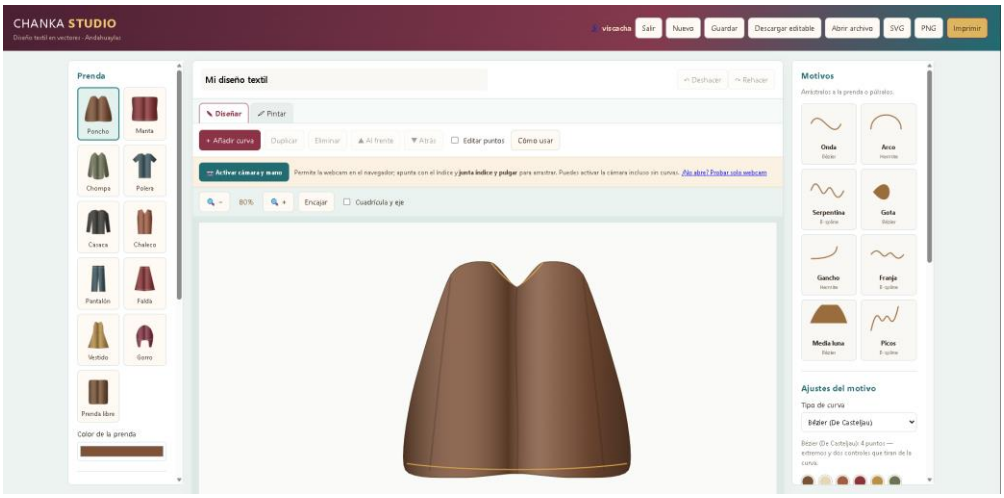
1. Descomprimir el repositorio o abrir la carpeta local del proyecto en Visual Studio Code.
2. Conservar la estructura de archivos y carpetas.
3. En Visual Studio Code, hacer clic derecho en index.html y seleccionar Open with Live Server. La extensión inicia un servidor local y abre la URL en el navegador.
4. Comprobar que se carguen la interfaz, las prendas y los paneles.

Para ejecutar la aplicación no se requiere una etapa de compilación indicada en el proyecto: se sirve index.html mediante Live Server. Mantener las rutas y carpetas originales para que se carguen correctamente CSS y JavaScript.

8.2 Ejecutar pruebas automatizadas

5. Instalar o disponer de Node.js en el equipo de desarrollo.
6. Abrir una terminal en la carpeta raíz del proyecto.
7. Ejecutar el comando: node tests/verificar.cjs
8. Guardar la salida de consola, registrar la versión de Node.js y anotar el código de salida.

La documentación del proyecto registra una ejecución que terminó con código 0 durante su preparación. Debe adjuntarse la captura real de la ejecución que corresponda a la entrega final.



9. PRUEBAS Y RESULTADOS

El archivo tests/verificar.cjs incluye pruebas de curvas y muestreo, transformaciones, inversa de matrices, punto dentro/fuera de un polígono, suavizado, copias, simetría, catálogo de prendas, operaciones del editor e historial. Estas pruebas comprueban lógica del programa; por sí solas no garantizan usabilidad ni compatibilidad completa con todos los navegadores.

PRUEBA	EVIDENCIA QUE DEBE ADJUNTARSE	ESTADO DOCUMENTADO
Curvas y muestreo	Captura de consola y resultado final.	Incluido en la suite; adjuntar salida real.
Matrices e inversa	Captura de consola y resultado final.	Incluido en la suite; adjuntar salida real.
Editor e historial	Prueba y captura de interacción.	Completar evidencia manual.
Archivos y exportación	Prueba de guardar, abrir y exportar.	Completar prueba manual.
Navegadores	Chrome, Edge o Firefox con versión registrada.	Completar prueba manual.
Rendimiento	Tiempo/FPS, equipo, navegador y cantidad de motivos.	Medición específica pendiente.

No se deben presentar porcentajes de CPU o memoria del sistema completo como si fueran métricas exclusivas de Chanka Studio. Para una medición de rendimiento válida, registrar el equipo, sistema operativo, navegador y versión, tamaño del diseño, número de motivos y repeticiones, tiempo de respuesta o FPS, y método de medición.

```
Curvas
  ✓ Bézier pasa por el primer y el último punto
  ✓ Bézier en t=0.5 de una cúbica da el punto esperado
  ✓ Hermite pasa por inicio y fin
  ✓ Hermite sale en la dirección de la tangente
  ✓ B-spline abierta: empieza y termina en los extremos
  ✓ B-spline: vector de nudos de 6 puntos
  ✓ B-spline con puntos en línea recta da una recta
  ✓ muestrear devuelve 101 puntos finitos

Matrices
  ✓ trasladar, escalar y rotar
  ✓ multiplicar aplica primero la matriz de la derecha
  ✓ la inversa deshace la transformación

Trazo y motivos
  ✓ dentro() distingue puntos dentro y fuera
  ✓ suavizar da un contorno cerrado con puntos finitos
  ✓ copiasMotivo: repeticiones y espejo
  ✓ el espejo refleja respecto al centro (x = 400)
  ✓ el giro mantiene el centro del motivo en (x, y)

Prendas
  ✓ prenda «poncho»: contorno válido
  ✓ prenda «manta»: contorno válido
  ✓ prenda «chompa»: contorno válido
  ✓ prenda «polera»: contorno válido
  ✓ prenda «casaca»: contorno válido
  ✓ prenda «chaleco»: contorno válido
  ✓ prenda «pantalon»: contorno válido
  ✓ prenda «falda»: contorno válido
  ✓ prenda «vestido»: contorno válido
  ✓ prenda «gorro»: contorno válido
  ✓ prenda «libre»: contorno válido
  ✓ la lista "orden" y el catálogo coinciden

Editor
  ✓ un diseño nuevo es válido
  ✓ agregar, duplicar y eliminar motivos
  ✓ deshacer y rehacer
  ✓ cambiar de prenda conserva los motivos
  ✓ contorno: agregar, quitar y restaurar puntos
  ✓ cambiar el tipo de curva restablece sus puntos

Archivos
  ✓ rechaza datos inválidos
  ✓ migra un diseño de la versión 1

Plantillas y pintura
  ✓ todas las plantillas producen diseños válidos
  ✓ pintar, borrar y deshacer un trazo
  ✓ alisar conserva extremos

39 correctas, 0 con error
```



10. LIMITACIONES CONOCIDAS

- Las prendas son esquemas visuales y no patrones de confección ni medidas reales.
- Los motivos geométricos no están certificados como representaciones históricas chanka.
- Las curvas se dibujan a partir de muestras discretas, con 100 pasos por curva.
- Las repeticiones se limitan a entre 1 y 10 por dirección, según la guía técnica.

- El guardado local puede perderse al limpiar los datos del navegador; se recomienda descargar JSON.
- El arrastre táctil y la interfaz deben verificarse manualmente en navegadores reales.
- La documentación disponible no incluye evidencia de entrevista o actividad RSU; debe añadirse por separado si corresponde.

11. MEJORAS POSIBLES

- Agregar pruebas automatizadas de exportación y de interacción de interfaz.
- Medir rendimiento con diseños pequeños, medianos y complejos en un equipo identificado.
- Mejorar accesibilidad y comprobar la navegación por teclado.
- Validar la interacción táctil y el comportamiento en distintos tamaños de pantalla.
- Recoger retroalimentación documentada de usuarios y priorizar mejoras según sus necesidades.
- Registrar las versiones exactas del navegador, Node.js y herramientas utilizadas.

12. REFERENCIAS TÉCNICAS

Eck, D. J. (2023). Introduction to computer graphics (Version 1.4). Hobart and William Smith Colleges. https://math.hws.edu/graphicsbook/

Guha, S. (2022). Computer graphics through OpenGL: From theory to experiments (4th ed.). Chapman and Hall/CRC.

Hearn, D., & Baker, M. P. (2006). Gráficos por computadora con OpenGL (3.ª ed.). Pearson Educación.